

네, 당신의 문제의식(초보자 아이디어를 시니어 개발자 수준 결과물로 **consistently** 고도화하는 방법론 자체를 연구하고, 그 방법론을 지속적으로 발전시키는 것)과 매우 유사한 프로젝트들이 2025~2026년에 활발히 진행되고 있습니다. 특히 **recursive self-improvement**, **output refinement loop**, **metacognitive self-modification**, **methodology evolution** 같은 키워드로 연구되고 있어요.

당신의 접근(메타-연구 + persistent learning + skill 축적 + reflection)은 현재 최첨단 에이전트 연구의 핵심 흐름과 잘 맞습니다. Hermes Agent의 built-in skill learning loop을 활용하면 이 연구들을 실무적으로 참고하기도 좋습니다.

가장 가까운 관련 프로젝트들 (참고 가치 순)

1. Darwin Gödel Machine (DGM) (2026, arXiv)

- 유사점: 에이전트가 스스로 코드를 수정하고, 개선된 버전을 empirical benchmark로 평가하며 archive(스킬/에이전트 저장소)에 축적. open-ended exploration으로 local optimum을 피함. 자기 개선 능력 자체를 개선 (metacognitive).
- 참고 포인트: 당신이 원하는 “고도화 방법론”을 에이전트가 스스로 진화시키는 구조. coding agent에 특화됐지만, refinement loop과 archive 관리 아이디어를 Hermes skill 시스템에 적용하기 좋음.
- GitHub/논문 쉽게 검색 가능.

2. HyperAgents (Meta, 2026)

- 유사점: Task Agent + Meta Agent 구조. Meta Agent가 Task Agent의 성능을 보고 코드를 직접 패치(patch)하며, 개선 메커니즘 자체도 수정 가능 (metacognitive self-modification).
- 참고 포인트: 단순 reflection이 아니라 “개선하는 방법론을 개선”하는 레벨. 당신의 메타-연구(방법론 자체 연구)에 가장 가까운 아키텍처. Hermes에 Meta-Evolor 역할할 때 영감이 될 수 있음.

3. EvoScientist (2026)

- 유사점: Multi-agent 시스템이 persistent memory를 통해 연구 전략(idea generation, validation, experiment strategy)을 스스로 진화시킴. 실패로부터 배우고 방법론을 개선.
- 참고 포인트: 과학 연구에 특화됐지만, “연구 프로세스 자체의 self-evolution” 부분이 당신의 목표와 거의 일치. Hermes의 multi-role (Researcher + Critic + Evolor) 구성에 직접 적용 가능.

4. The AI Scientist (v1 & v2) (Sakana AI, 2024~2025)

- 유사점: 가설 생성 → 실험 설계 → 코드 실행 → 분석 → 논문 작성까지 end-to-end. agentic tree-search와 iterative refinement, automated reviewer로 출력 품질을 높임.
- 참고 포인트: 결과물(논문/코드)을 고도화하는 구체적인 refinement 기법. v2에서는 VLM 피드백 루프 추가로 시각화/설명 품질도 개선. 당신이 “산출물 고도화 방법론”을 테스트할 때 좋은 벤치마크.

5. Agent Laboratory

- 유사점: 인간이 아이디어를 주면 문헌 검토 → 실험 → 보고서까지 자동화. 여러 specialized agent 협업.
- 참고 포인트: 초보자 아이디어를 고품질 결과물로 만드는 워크플로우 전체를 agentic하게 구현한 사례.

기타 실무적 참고:

- **Self-Refine / Reflexion** 계열: Generate → Critique → Refine 반복 루프 (가장 기본적이면서 강력함).
- **Karpathy의 AutoResearch loop**: 평가 metric 정의 → 자동 실험 → 결과 리뷰 → 반복. prompt/skill 개선에 자주 쓰임.
- **Hermes Agent 자체**: 당신이 이미 쓰고 있는 Hermes가 “built-in self-improving skill system”으로 유명합니다. 다른 프로젝트들의 아이디어를 Hermes skill + memory에 녹여서 구현하면 시너지가 큼.

이 연구들을 어떻게 참고해서 진행할까? (실천적 조언)

- 단계별 참고 전략:

1. 이론/구조 참고: HyperAgents와 Darwin Gödel Machine 논문을 먼저 읽으세요. Meta Agent(또는 Evolution Manager) 역할을 Hermes에 추가하는 설계에 큰 도움이 됩니다.
2. 워크플로우 참고: EvoScientist와 AI Scientist의 multi-agent 팀 + persistent memory + evolution mechanism을 Hermes의 skill 생성/개선 루프에 매핑.
3. **Refinement** 기법 참고: Self-Refine + agentic tree-search를 당신의 초기 프롬프트에 넣어 “Generate → Evaluate (rubric 기반) → Critique → Refine”을 여러 iteration 돌리게 하세요.
4. 평가 체계: AI Scientist의 automated reviewer나 LLM-as-a-Judge를 도입해 “시니어 수준”을 객관적으로 측정하게 하세요.

- **Hermes**에 적용하기 좋은 포인트:

- Skill 라이브러리에 “Refinement Methodology v1”, “Meta-Evolution Pattern” 같은 dedicated skill 을 만들어 축적.
- 매 연구 사이클 끝에 “이번 refinement 방법론을 vN+1로 업그레이드하라” 지시.
- Context rot 방지를 위해 structured memory (Goal, Decisions, Learned Patterns, Next Improvement) 형식 강제.
- 주의점: 대부분의 프로젝트가 coding/scientific domain에 치중되어 있어요. 당신의 “초보자→시니어 개발자 결과물”은 소프트웨어 엔지니어링에 더 가깝기 때문에, 논문 아이디어를 **Hermes skill**과 **Human-in-the-Loop** 피드백으로 보완하며 커스터마이징하는 게 가장 현실적입니다.

전체적으로 당신의 문제의식은 **2026년 AI 에이전트 연구의 hottest topic** 중 하나와 정확히 일치합니다. “단순 결과물 생성”을 넘어 “고도화 방법론 자체를 진화시키는 에이전트”를 만드는 건 매우 의미 있는 방향이에요.